

PLAN.md – LMS

Modernization Master Plan (v4 Final)

You are a senior enterprise Java architect and implementer working on the BajajLife LMS backend modernization. This document is your single source of truth. Follow it exactly. Do not expand scope beyond it. When in doubt, choose the SIMPLER option and log the question in `DECISIONS-PENDING.md` instead of inventing a solution.

This plan is designed to remain stable for the entire modernization. Future updates will only APPEND module mapping details — the architecture, conventions, and rules below do not change.

1. Validated current state (from codebase analysis)

- AS-IS: single WAR `BajajLMS.war` on WebLogic 12c cluster, Spring Boot 2.7.1 / JDK 1.8, ≈400 endpoints under `/BalicLmsUtil/*` (Lead 22, IB 28, Agency 30, Other 38, Inhouse 13, Common 14,

Admin 10, Dashboard 10, Report 8, SubAdmin 3, DigitalAsset 1, ExternalService 2 controllers).

- God classes: `CommonServiceImpl` 3,480 LOC (serves all domains), `IBLeadServiceImpl` 2,778 LOC, `AgencyServiceImpl` 1,287 LOC, `LeadSqlBuilder` 723 LOC dynamic Oracle SQL, ~60 other `ServiceImpls`, ~210 JPA repositories.
- Data: PostgreSQL `batracker` ~120 entities WRITABLE; Oracle OPUS/MISDEV `CUSTOMER` ~70 entities READ-MOSTLY. No XA / no JTA – cross-DB consistency is already best-effort. **Schemas MUST NOT change. No data migration.**
- Downstream systems: BimaSugam, IManage SSO, Customer360, SmartPitch, SlashRTC telephony, FCM, Bitly, Eureka CRM, iRecruit, Lemnisk DMP/SMS, RealTime Comms (CCM), SMTP.
- Partner auth today: TPA/BFL/NBM vendors and WebSales `/pushData` use a key+secret+encrypted-header scheme.
- Domain model (validated against code): D1 Lead Core (hub; owns `azbj_lms_lead_dtls`), D2 IB/Bank Channels, D3 Agency/SAM, D4 Inhouse+WebSales, D5 DSR/PSF Reporting, D6 Renewal/Retention, D7 Medical/Recruitment, D8 Specialized Programs, D9 Master Data, D10 Identity/Access/Audit, D11 Bulk Uploaders, D12 Integration Gateway. Dependency shape is hub-

and-spoke: all domains read/update D1, lookup D9, authz via D10.

- Consumer: ONE new common app. Target: ~40–50 consolidated, well-documented APIs replacing the 400+.
- Platform context: deployment to OpenShift (OCP) on AWS via GitHub Actions CI/CD. 3scale API gateway and RHSSO (token issuance) already exist in the estate; the app will be onboarded to 3scale AFTER modernization completes.
- Maintainers: junior Java developers. All lead/customer data is PII.

2. Locked architecture decisions (never revisit)

1. **Modular monolith. ONE deployable Spring Boot executable JAR**, containerized, running on OCP. NOT a WAR, no WebLogic anywhere in the new stack. No microservices, no in-app gateway, no Kafka/Redis/S3, no Vault buildout, no dual-write, no service extraction. (The 12-month strangler roadmap is a future vision document — never implement any of it.)
2. **Java 21 LTS + Spring Boot 3.5.x + Spring Modulith** for build-time module boundary enforcement.
3. **Virtual threads enabled**
(`spring.threads.virtual.enabled=true`). Plain

blocking code everywhere. No WebFlux, no reactive, no CompletableFuture chains unless ported as-is from old code.

4. **Database stays exactly as-is.** Entities map existing tables verbatim. Entities never leave the `infrastructure` layer.
5. **Oracle quarantine.** All Oracle access through read-model adapter classes in each module's `infrastructure/oracle/`, returning clean DTOs. `LeadSqlBuilder` is PORTED AS-IS into `leadcore/infrastructure/oracle/` behind an interface — do NOT rewrite it to JPA/QueryDSL.
6. **God classes are dismantled, never ported wholesale.** Each module harvests ONLY the methods it uses from `CommonServiceImpl / ILeadServiceImpl` /etc. into its own application services. There is NO shared business-logic module — `shared` contains technical plumbing only.
7. **Reuse, don't rewrite.** Harvest existing entities, repositories, and business logic. Ambiguous logic is ported as-is with `// PORTED-AS-IS: verify` plus a `DECISIONS-PENDING.md` entry.
8. **Resilience4j on every downstream/external call:** connect timeout 2s, read timeout 5s, circuit breaker, fallback returning the standard error envelope with a specific code (e.g.

`EXT_CRM_UNAVAILABLE`). A request must never hang on a downstream.

9. **API-first.** Every module's consolidated endpoints are defined in an OpenAPI spec and approved BEFORE implementation. springdoc-openapi serves live docs. URL convention designed for painless 3scale onboarding later:

`/api/v1/{resource}` — versioned from day one, consistent error envelope, standard health endpoints.

3. Code style rules (junior-friendly, conservative — MANDATORY)

- **Classic POJOs with Lombok** (`@Getter` `@Setter` `@NoArgsConstructor` `@AllArgsConstructor` `@Builder` as appropriate) for ALL DTOs, domain objects, and entities. **Do NOT use Java records for DTOs or models.** Records are permitted only for tiny internal value holders (e.g., a private map key), and even then prefer a POJO if in doubt.
- **Do NOT showcase JDK 21 features.** No sealed classes/interfaces, no pattern-matching-for-switch demonstrations, no var overuse, no text-block tricks. Virtual threads (config-level) and straightforward simplifications (enhanced switch where it clearly reads better) are fine.

- Boring code is correct code. Explicit over clever: simple if/else over nested ternaries, plain loops over complex stream chains (simple `stream().map().collect()` is fine; nested/multi-collector streams are not).
- Some static utility code is acceptable (e.g., a `MaskingUtils`, `DateUtils` in shared) — keep utilities stateless and small.
- Methods $\leq \sim 40$ lines; extract private methods aggressively. A junior must be able to read any class top-to-bottom.
- Constructor injection only (Lombok `@RequiredArgsConstructor`). No field injection. No `@Autowired` on fields.
- One pattern repeated everywhere: controller $\rightarrow$ use-case service $\rightarrow$ repository/adapter $\rightarrow$ mapper. No module deviates.
- MapStruct for mapping (entity $\leftrightarrow$ domain $\leftrightarrow$ DTO). Mappers are interfaces — generated code keeps juniors out of hand-written mapping bugs.
- Javadoc on every public application-service method: one line saying what it does in business terms + which old API(s) it replaces.

4. Production-grade baseline (mandatory, thin)

4.1 Security (two mechanisms, both standard)

- **RHSSO (Keycloak) JWT validation** via Spring Security OAuth2 Resource Server:
`spring.security.oauth2.resourceserver.jwt.issuer-uri` per profile. Roles mapped from RHSSO realm/client roles to Spring authorities with a small converter. NO custom token code. The new common app calls with RHSSO-issued bearer tokens.
- **Partner key+secret+encrypted-header filter** for partner-facing intake endpoints (`/pushData` -style):
PORT the existing validation logic from the old WAR as a Spring Security filter on those routes only. Existing partner credentials keep working unchanged.
- `@PreAuthorize` role checks on every command (write) endpoint.
- Bean Validation (`@Valid` + constraints) on every request DTO. Unknown JSON fields rejected.
- **No PII in logs**: never log request/response bodies; mask phone/email in any diagnostic log via `MaskingUtils` . Documented in CONVENTIONS.md.
- No stack traces in API responses. Actuator: health/info public; all other endpoints authenticated.
- 3scale note: app-level auth stays as above; 3scale will add edge API-key/rate-limit policies at

onboarding time – requires zero app code change because paths are versioned and stable.

4.2 Observability (pragmatic floor)

- Structured JSON logging (logstash-logback-encoder) to stdout (Grafana/Loki-compatible on OCP) with a correlation-ID servlet filter; the ID is returned in every response header and present in every log line.
- Spring Boot Actuator: `health` (with liveness checks for BOTH datasources), `metrics`, `info`. These also serve as OCP readiness/liveness probes.
- Micrometer `@Timed` on every application-layer use-case method.
- No distributed tracing in this modernization (single runtime – stack traces work).

4.3 Configuration & environments

- Spring profiles: `local`, `sit`, `uat`, `prod`. ZERO secrets in Git – all credentials via environment variables backed by OCP Secrets. Provide `application-local.yml.example`.
- `docker-compose.yml` for local development: app + PostgreSQL. Oracle via dev-instance connection, or `oracle.enabled=false` stub flag making Oracle adapters return a typed `NotAvailable` response.

4.4 Database discipline

- Flyway from day one. New-app-owned objects ONLY (e.g., `lmsng_idempotency_key` table), prefixed `lmsng_`. Business schema is never touched.
- Two bounded HikariCP pools (Postgres, Oracle) with explicit `maximum-pool-size` (start: 20 each) and `connection-timeout`. CRITICAL with virtual threads – unbounded concurrency must meet bounded pools deliberately. Numbers documented in CONVENTIONS.md.
- Every list endpoint is paginated. No unbounded `findAll`. Watch N+1 when consolidating multi-API reads into one endpoint – use joins or `@EntityGraph`, never sequential repository loops.

4.5 CI/CD (simple, two workflows)

- **ci.yml** (on pull request): JDK 21 setup → `mvn verify` (unit + Testcontainers integration tests) → build fails on test failure. Nothing else.
- **cd.yml** (on merge to main): build executable JAR → build container image (Containerfile, base `registry.access.redhat.com/ubi9/openjdk-21`) → push to image registry → `oc apply /rollout` to SIT namespace on OCP.
- Plain OCP Deployment + Service + Route YAML in `/deploy`. NO Helm, NO ArgoCD, NO multi-stage promotion pipelines, NO IaC frameworks.

Promotion to UAT/prod is a manual, documented
oc step for now.

- OCP probes wired to actuator health; resource requests/limits set conservatively (start: 1 CPU / 2Gi, tune later).

5. Module structure (template — identical for every module)

```
com.bajajlife.lms
├─ shared/                → technical
plumbing ONLY: error envelope, ApiResponse,
|                          global
exception handler (@RestControllerAdvice),
|                          correlation-
ID filter, security config (RHSSO resource
|                          server +
partner-key filter),
logging/masking/validation
|                          utils,
Flyway + dual datasource config. NO business
logic.
├─ masterdata/
|   └─ api/                →
@RestController classes + request/response
DTO POJOs
|   └─ application/        → use-case
services; @Transactional and @Timed live
HERE
|   └─ domain/             → business
objects, enums, rules (plain Java, no
```

```

Spring/JPA)
|   └─ infrastructure/
|       └─ postgres/           → harvested
JPA entities + Spring Data repositories
|       └─ oracle/             → read-model
adapters (JdbcTemplate / ported SQL
builders)
|       └─ external/           → downstream
clients wrapped in Resilience4j
|       └─ mapper/             → MapStruct
mapper interfaces
|       └─ leadcore/           → same
structure; ADDITIONALLY exposes the public
|                               application
interface LeadCoreService – the ONLY way any
|                               other module
reads/updates leads (Modulith-verified)
|       └─ inhouse/           → same
structure
└─ ... (every domain module clones this
exactly)

```

Hard rules:

- Cross-module access ONLY via public application services (Spring Modulith verification test enforces this in CI).
- Standard response wrapper: `ApiResponse<T> { data, error{code, message}, traceId, timestamp }`. Pagination: `page, size, totalElements, totalPages`.

- Ugly legacy column names are mapped to clean field names ONCE, in mappers. The API contract never exposes legacy naming.
- One global exception handler in `shared` ; modules throw typed exceptions, never build error responses.

6. Milestones (sequence matters; calendar is flexible)

M0 — Inventory, interface design & foundation

1. Scan the old codebase → `inventory.csv` : endpoint path, HTTP method, controller, service class(es), Postgres tables, Oracle tables, downstream systems, complexity (S/M/L).
2. **God-class harvest map**: for `CommonServiceImpl` and `IBLeadServiceImpl` , map every public method → owning target module. For methods used by multiple modules, propose an owner + public interface; present options, do not decide alone.
3. **Design the `LeadCoreService` public interface NOW** (interface + DTOs only, no implementation): the lead read/update operations every other module needs. This is the system's most important boundary.

4. Propose the consolidated endpoint list (OpenAPI) for D9, D1, D4 with old → new mapping (target 4–8 endpoints per module). **STOP — present #2, #3, #4 together for human approval.**
5. Build the foundation: project skeleton (Modulith structure, all module packages with package-info), `shared` fully implemented (both security mechanisms, correlation filter, envelope, exception handler, actuator, JSON logging, Flyway, dual datasources with bounded pools), `CONVENTIONS.md` capturing every rule in sections 3–5, profiles, docker-compose, ci.yml + cd.yml + Containerfile + /deploy YAML.

M1 — Pilot: D9 Master Data

Implement per approved mapping (harvest entities/repos → consolidated use-case services → controllers per OpenAPI contract → mappers). Tests: Testcontainers (Postgres) integration test per endpoint; Modulith boundary verification; characterization tests comparing new responses to captured old-API responses for top-traffic APIs. **STOP — human review. This module is the frozen template for everything after.**

M2 — Core: D1 Lead Core

Implement `LeadCoreService` + its REST endpoints. Port `LeadSqlBuilder` AS-IS behind an

`OracleLeadReadModel` interface. Every multi-table write that old consumers performed via sequential API calls becomes ONE command endpoint with a single `@Transactional` boundary — list these consolidations explicitly for review. Command endpoints accept an `Idempotency-Key` header backed by `lmsng_idempotency_key` (duplicate within 24h returns the original response). Characterization tests: ALL write endpoints + top-20 reads.

M3 — Intake: D4 Inhouse + WebSales

Same template. Partner-facing `/pushData` -equivalent endpoints secured by the ported key+secret filter. All downstream pushes (CRM, FCM, SMS, telephony, etc.) via `infrastructure/external` clients with Resilience4j. This proves the `LeadCoreService` boundary (inhouse consumes it) and the partner-auth pattern.

M4 — Replication: remaining domains in this order

1. D2 IB/Bank Channels (largest — benefits from mature template; dismantle `IBLeadServiceImpl` per the harvest map)
2. D3 Agency/SAM
3. D10 Identity/Access/Audit (until then, `shared` security covers the minimal authz the built modules need)

4. D11 Bulk Uploaders
5. D6 Renewal/Retention
6. D7 Medical/Recruitment
7. D8 Specialized Programs
8. D5 DSR/PSF Reporting
9. D12 Integration Gateway (mostly absorbed into per-module `infrastructure/external` — implement only what remains genuinely cross-cutting, e.g., shared notification templates)
Each module: same template, same test bar, one module per session, review gate after each.

M5 — Hardening & handover

Swagger descriptions + sample requests for every endpoint; README (one-command local run, dual-DB config); seed/test data scripts; `KNOWN-GAPS.md` ; consolidated `DECISIONS-PENDING.md` ; test charter mapping every new endpoint → old APIs replaced; load sanity check on top-10 endpoints (virtual threads + pool sizing validated under concurrency).

M6 — Cutover & 3scale readiness

`CUTOVER.md` : reads may overlap between old WAR and new app; **write ownership of a table cluster moves atomically with its module go-live** (the consuming app switches routes per module; writes are NEVER split between old and new apps for the same tables). Per-

module decommission of old APIs based on traffic evidence. 3scale onboarding checklist: register `/api/v1/*` paths, map RHSSO issuer, edge policies — config only, no app change.

7. Working rules for you (the AI) — every session

- One module per session. Never modify two modules in one change set.
- Later modules follow the M1 pilot byte-for-byte. Consistency beats cleverness — always.
- Never port a god class wholesale; harvest per the approved map.
- Never invent business logic; never touch the business schema; never "fix" denormalization; never convert query styles (JPA stays JPA, JdbcTemplate stays JdbcTemplate, ported SQL builders stay as-is).
- Never start anything from the strangler roadmap (gateway-in-app, K8s platform work beyond /deploy YAML, Kafka/Redis/S3, Vault, extraction waves, dual-write/shadow-read).
- Respect section 3 code style in every file: POJOs + Lombok, no records for DTOs/models, no JDK 21 feature showcasing.
- After each milestone output: what was built, files changed, items needing human review, open

questions appended to DECISIONS-PENDING.md.

8. Definition of done

Per module: endpoints match approved OpenAPI; integration + characterization tests green; Modulith verification green; conventions followed (spot-check: no records as DTOs, constructor injection, paginated lists, Javadoc on use-case methods).

Overall: one-command local run; executable JAR container deploys to OCP SIT via cd.yml; Swagger UI lists all modules; actuator health green for both datasources; both auth mechanisms enforced (invalid RHSSO token → 401; partner endpoints validate key+secret); JSON logs with correlation IDs and zero PII; ~40–50 consolidated endpoints with complete old → new mapping; a new module can be added by cloning the template with ZERO new architectural decisions.

9. Explicitly rejected (do not re-propose)

Microservices / service extraction; API gateway inside the app; Kafka, Redis, S3, Vault; dual-write or shadow-read migration; LeadSqlBuilder rewrite to JPA/QueryDSL; WebFlux/reactive; records-based DTO layer; sealed-type domain modeling; Helm/ArgoCD/IaC

frameworks; distributed tracing; caching layers; event-driven refactors; schema normalization. Each of these may have future merit — none belongs in this modernization.